RUBY-GPT - Flat File Internet Search Chat-Bot

RUBY-GPT is a JOHTML single-page website that behaves like a sophisticated AI chat-bot while using open internet search as its answer source. It does not need MySQL, PostgreSQL, Redis, or a vector database. It stores every user prompt and answer in flat files.

What RUBY-GPT does

1. The user enters a prompt in `INPUT_CHAT`.
2. The user selects a `GPT_ENGINE`, `GPT_MODEL`, and `COMPLEXITY`.
3. The frontend sends the request to the PHP backend.
4. The PHP backend performs an open internet search using DuckDuckGo HTML search.
5. The backend opens the first result URL.
6. The backend extracts readable text from that page.
7. The answer is displayed in an editable, scrollable modal.
8. The request and answer are stored as flat files.

Stack

• HTML5 single-page website

• PHP 8.3 backend

• Apache web server

• Bootstrap 5

• Tailwind CSS

• Font Awesome

• Docker Compose

• Flat file storage using JSON and TXT

Project structure

```
ruby-gpt-flatfiles/
■■■ api/
■   ■■■ search.php       # Search endpoint that fetches first open web result
■   ■■■ history.php      # Reads saved flat file answer history
■■■ storage/
■   ■■■ requests/        # Prompt JSON/TXT files
■   ■■■ answers/         # Answer JSON/TXT files
■   ■■■ logs/            # Error logs
■■■ index.php            # Single-page HTML5 website
■■■ Dockerfile           # PHP Apache image
■■■ docker-compose.yml   # docker compose up solution
■■■ README.md            # Main documentation
■■■ USER-GUIDE.md        # Bash creation and run commands
```

Run with Docker Compose

```
docker compose up --build
```

Open the app:

```
http://localhost:8080
```

Using the application

1. Open `http://localhost:8080`.
2. Go to the `CHAT` section.
3. Enter your prompt in `INPUT_CHAT`.
4. Choose a `GPT_ENGINE` such as `CHATGPT`, `GROQ`, `GEMINI`, or `CLAUDE`.
5. Choose the associated `GPT_MODEL`.
6. Choose the `COMPLEXITY` level.
7. Click `PROCESS`.
8. Review the answer inside the editable modal.
9. Click `LOAD FLAT FILE HISTORY` to view saved answer records.

Flat file storage details

Each request is stored in:

```
storage/requests/<id>.json
storage/requests/<id>.txt
```

Each answer is stored in:

```
storage/answers/<id>.json
storage/answers/<id>.txt
```

If a backend error occurs, it is stored in:

```
storage/logs/<id>.error.log
```

API endpoints

POST `/api/search.php`

Request body:

```
{
  "INPUT_CHAT": "Explain Ruby on Rails Active Storage with Azure Blob Storage",
  "GPT_ENGINE": "CHATGPT",
```

```
  "GPT_MODEL": "gpt-4.1",
  "COMPLEXITY": "LEVEL"
}
```

Response body:

```
{
  "ok": true,
  "id": "20260516_120000_ab12cd34",
  "created_at": "2026-05-16T12:00:00+00:00",
  "engine": "CHATGPT",
  "model": "gpt-4.1",
  "complexity": "LEVEL",
  "source_title": "Example Result",
  "source_url": "https://example.com",
  "answer": "RUBY-GPT ANSWER..."
}
```

GET `/api/history.php`

Returns the latest 25 flat file answer records.

Important implementation note

RUBY-GPT does not call paid LLM APIs. The engine and model fields are a routing-style UI profile. The answer is obtained from the first open internet search result. This makes the project useful as a low-cost search-powered chatbot prototype.

Troubleshooting

The answer modal shows an error

Possible causes:

- The container has no internet access.
- DuckDuckGo blocked or changed its HTML result format.
- The first result blocks automated fetching.
- The first result has no readable HTML text.

Try another query or modify `api/search.php` to use another open search provider.

Permission denied writing to storage

Run:

```
chmod -R 775 storage
```

If needed:

```
sudo chown -R $USER:$USER storage
```

Port 8080 already in use

Edit `docker-compose.yml`:

```
ports:
  - "8090:80"
```

Then open:

```
http://localhost:8090
```

Security notes

• This project does not execute user commands.

• Input length is limited in `api/search.php`.

• Production deployments should add rate limiting, CSRF protection, URL allow/deny controls, caching, and stricter content extraction.

Upgrade ideas

• Add real API routing to OpenAI, Claude, Gemini, Groq, and Perplexity.

• Store page snapshots in flat files.

• Add a local inverted index over stored answers.

• Add summarisation using a selected LLM provider.

• Add source citations for multiple search results.

• Add admin login for browsing flat file records.